

ESERCIZI...

→ RICORRENZA

$$a(n) + b \Rightarrow a(n+1) + b$$

Domanda 5 Risolvere la ricorrenza

$$T(n) = \begin{cases} 3 & \text{se } n = 0 \\ T(n-1) + 2 & \text{se } n > 0 \end{cases}$$

utilizzando il metodo di sostituzione per determinare una soluzione esatta (non asintotica).

Soluzione: Mostriamo per induzione che $T(n) = an + b$, determinando a e b . Per $n = 0$ si ottiene $T(0) = 3 = b$, quindi $b = 3$. Nel caso induttivo, si ottiene

$$T(n+1) = T(n) + 2 = an + b + 2$$

Affinché $an + b + 2 = a(n+1) + b$ occorre che $a = 2$. Quindi, $T(n) = 2n + 3$.

$$an + 2 = 3$$

$$1 \cdot 1 + 2 = 3$$

$$3 = 3$$

$$T(n-1+1) \quad n=1, a=1$$

$$+ 2 = \frac{T(n)+2}{3} = \frac{an+b}{3} = 3$$

$$- n=0 \Rightarrow T(n) = a \cdot 0 + \frac{b}{3} = 3$$

$$- n \rightarrow n+1$$

$$T(n+1) = T(n-1+1) + 2 \\ = T(n) + 2 = an + b + 2$$

$$b=3 \rightarrow an + 2 = 3$$

$$a=1, n=1 \quad \nearrow \frac{1}{3}$$

$$T(n) = 2n + 3$$

$$T(n-1+1) + 2 \\ = T(n) + 2$$

$$(T(n-1+1)+2=T(n)+2$$

$an+b+2=a(n+1)+b \rightarrow$ caso induttivo. Trovo $a = 2$ e con $b=3$ formo $T(n) = 2n + 3$

$$\frac{a(n+1)}{1} + \frac{3}{1}$$

Domanda 6 Sia data la seguente equazione di ricorrenza:

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 3T(n/5) + T(n/6) + n & \text{se } n > 1 \end{cases}$$

Si fornisca un limite asintotico stretto per la soluzione.

Soluzione: Vale $T(n) = \Theta(n)$. Dimostriamo ad esempio che

$$T(n) = \Omega(n)$$

CASO 3 $\rightarrow$

REGOLARITÀ

Nota: quando c'è una ricorrenza di questo tipo (composta da uno o più termini), dobbiamo capire a che cosa il limite asintotico è stretto. Basta vedere, quasi sempre, il termine noto. Asintoticamente, tutto tende a quello.

$$\frac{n}{1} \Rightarrow n+1$$

CASO BASE

$$\Theta \subset \Omega$$

GRAFICA + USO DI SOSTITUZIONI

$$\Rightarrow n=0 \Rightarrow T(n) = \frac{a(n)}{b} + 1 = 1$$

$$\Rightarrow (n \Rightarrow n+1) T(n)$$

$$3T\left(\frac{n}{5}\right) + T\left(\frac{n}{6}\right) + n$$

$$\Rightarrow \left(3T\left(\frac{n+1}{5}\right) + T\left(\frac{n+1}{6}\right) + n+1 \right)$$

$$T(n+1) = a(n+1) + \underbrace{b}_1$$

RISOLVO PER "a"
e per "n"

$$= -1$$

$$a f\left(\frac{n}{b}\right) \leq k \cdot f(n) \quad \equiv \quad \text{CASO 3}$$

Domanda 10 Si consideri la ricorrenza

$$T(n) = T\left(\frac{4n}{5}\right) + \frac{n}{2} + \log n$$

Fornire limite asintotico stretto per la soluzione.

Caso 3: $f(n) \rightarrow \log(n)$ vs $n \log_b(a) \rightarrow 1$
Deve quindi valere la condizione di regolarità, risolvendo per k .
Quindi, il limite stretto è per $\theta(n \log_b(a)) \rightarrow \theta(n)$

Soluzione: Si può utilizzare il master theorem dato che la ricorrenza è nella forma

$$T(n) = a T\left(\frac{n}{b}\right) + f(n)$$

con $a = 1$, $b = 5/4$ e $f(n) = n/2 + \log n$.

Vale che $n^{\log_b a} = n^{\log_{5/4} 1} = n^0 = 1$, quindi

$$f(n) = \Omega(n^{\log_b a + \epsilon}) = \Omega(n^\epsilon)$$

per $0 < \epsilon < 1$.

Inoltre vale la condizione di regolarità $a f(n/b) \leq k f(n)$ per qualche $0 < k < 1$. Infatti

$$\begin{aligned} a f(n/b) &= f(4n/5) \\ &= 2n/5 + \log 4n/5 \\ &= 2n/5 + \log n - \log(5/4) \\ &< k(n/2 + \log n) \end{aligned}$$

che risulta soddisfatta per $k > 4/5$ (quindi $k/2 > 2/5$) e n abbastanza grande. Quindi $T(n) = \Theta(n + \log n) = \Theta(n)$.

Domanda 12 Dare la definizione di $\Omega(f(n))$. Dimostrare che la ricorrenza che segue ha soluzione
 $T(n) = \Omega(2^n)$

$$T(n) = \sum_{k=1}^{n-1} T(k) T(n-k)$$

$$C \cdot 2^k \cdot C \cdot 2^{n-k}$$

Soluzione: Si deve provare che esiste una costante $c > 0$ e un n_0 tali che per ogni $n \geq n_0$

$$T(n) \geq c 2^n \Rightarrow \underline{\quad} \Rightarrow \underline{\quad}$$

Si procede induttivamente:

$$\begin{aligned} T(n) &= \sum_{k=1}^{n-1} T(k) T(n-k) \\ &\geq \sum_{k=1}^{n-1} c 2^k c 2^{n-k} \\ &= \sum_{k=1}^{n-1} c^2 2^n \\ &= c^2 (n-1) 2^n \\ &\geq c 2^n \end{aligned}$$

Qui ci sta una serie, ma non ci spaventiamo.

Banalmente, basta sostituire al termine della serie 2^n dove appare n e 2^k negli altri casi (dove ci sta $-k$, si mette $2^{-(k)}$).

Quando siamo nel punto "finale" di una serie, si mette sempre l'argomento (c^2) moltiplicato per quello a cui tende la serie $(n-1)$. A quel punto, abbiamo risolto: si trovano solo dei valori che rendono vera la disuguaglianza trovata, cioè: $c^2 2^{(n-1)} \geq c 2^n$, che vale appunto (si prova con dei numeri a caso) per $c \geq 1$ e per $n \geq 2$

per $n \geq 2$, e qualunque $c \geq 1$.

SSNTVZIONS / INDUZIONI $\rightarrow$ 1 %!
STRETTO / $\rightarrow$ 99 %.
(ESSNBA GRATTA)

MT — FUNCTIONS $\rightarrow$ DIVERSA SOL.
 $\downarrow$

$$Fib(2) = Fib(1) \cdot \frac{n+1}{3}$$

- CASO BASE
- CASO RICORSIVO

$$\theta \begin{cases} \Omega \\ \omega \end{cases}$$

IND. FALLISOS ! (SBAGLIA LA CONSEGNA...)

$$T(n) \leq C \cdot n$$

$$Cn+1 \leq Cn \rightarrow \underline{\text{NO!}}$$

$$\underline{T(n-1) \leq C \cdot n - 1} \quad (\underline{\text{IND. FORM.}})$$

↓
1/2

↓

MT $\rightarrow$ ~~PROVA~~ ...

Domanda 17 Dare la definizione di $\Omega(f(n))$. Mostrare che se $f(n) = \Omega(g(n))$ e $g(n) = \Omega(h(n))$ allora $f(n) = \Omega(h(n))$.

Nei seguenti due casi, si devono dimostrare le condizioni date usando le definizioni formali (con i limiti).

$f(n) = \Omega(g(n)) \rightarrow 0 \leq c_1 g(n) \leq f(n)$

$g(n) = \Omega(h(n)) \rightarrow 0 \leq c_2 h(n) \leq g(n)$

Metto insieme e ottengo che: $0 \leq c_2 h(n) \leq c_1 g(n) \leq f(n)$ (con $h(n)$ sotto dato che poi uso $g(n)$ anche per f)

Soluzione: Si ha che

$$\underline{\Omega(f(n)) = \{g(n) \mid \exists c > 0. \exists n_0 \in \mathbb{N}. \forall n \geq n_0. 0 \leq cg(n) \leq f(n)\}.$$

Se $f(n) = \Omega(g(n))$ e $g(n) = \Omega(h(n))$ allora esistono $c_1, c_2 > 0, n_1, n_2 \in \mathbb{N}$ tali che per ogni $n \geq n_1$

$$0 \leq c_1 g(n) \leq f(n) \quad (1)$$

e per ogni $n \geq n_2$

$$0 \leq c_2 h(n) \leq g(n) \quad (2)$$

Ne consegue che per ogni $n \geq \max\{n_1, n_2\}$, moltiplicando (2) per c_1 si ha

$$0 \leq c_1 c_2 h(n) \leq c_1 g(n) \leq f(n)$$

ovvero, indicato con $n_0 = \max\{n_1, n_2\}$ e $c = c_1 c_2$, per ogni $n \geq n_0$,

$$0 \leq ch(n) \leq f(n)$$

ovvero $f(n) = \Omega(h(n))$.

Qui metto insieme le costanti c_1 e c_2 tali che valga per tutte le funzioni considerate, partendo da $h(n)$.
"Togliendo di mezzo" c_1 e c_2 (metto una costante sola, tale che sia il massimo e quindi $g(n)$ diventa "inclusa" in $h(n)$ $\rightarrow$ essendo che metto una costante sola e $h(n)$ ne ha due, $g(n)$ diventa trascurabile.
A quel punto, notiamo che $h(n)$ è un limite inferiore (omega).

FUNZIONI $\rightarrow \dots$

PROG.DIN. / GREEDY

Esercizio 2 (10 punti) Abbiamo n programmi da eseguire sul nostro computer. Ogni programma j , dove $j \in \{1, 2, \dots, n\}$, ha lunghezza ℓ_j , che rappresenta la quantità di tempo richiesta per la sua esecuzione. Dato un ordine di esecuzione $\sigma = j_1, j_2, \dots, j_n$ dei programmi (cioè, una permutazione di $\{1, 2, \dots, n\}$), il tempo di completamento $C_{j_i}(\sigma)$ del j_i -esimo programma è dato quindi dalla somma delle lunghezze dei programmi $j_1, j_2, \dots, j_i$. L'obiettivo è trovare un ordine di esecuzione σ che minimizza la somma dei tempi di completamento di tutti i programmi, cioè $\sum_{j=1}^n C_j(\sigma)$.

- (a) Dare un semplice algoritmo greedy per questo problema, e valutarne la complessità.
(b) Dimostrare la proprietà di scelta greedy dell'algoritmo del punto (a), cioè che esiste un ordine di esecuzione ottimo σ^* che contiene la scelta greedy.

① ORDINO

② IF = CONDIZIONE GREEDY

③ CUT AND PASTE



SCelta FUORI
DALL'ATTUALE SOL.

$$C^* \Rightarrow C^* + 1$$

$$C^* = C^* \cup \{2, \dots, n\}$$

$$C^1 = C^* \setminus \{2, \dots, n\}$$



CUT AND PASTE CON SOL. OTTIMA

$\Rightarrow$ ASSUNDO!



SPUNTI DIN. GREEDY $\rightarrow$

§ 152,
35/36
MODUS

$$l(i, j) = |LCS(X_i, Y_j)|$$

$$l(i, j) = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 \quad (\text{caso 0}) \\ l(i-1, j-1) + 1 & \text{se } i, j > 0 \text{ e } x_i = x_j \quad (\text{caso 1}) \\ \max\{l(i, j-1), l(i-1, j)\} & \text{se } i, j > 0 \text{ e } x_i \neq x_j \quad (\text{caso 2}) \end{cases}$$

Alla fine ci interessa calcolare $l(m, n)$.

Per calcolare $l(i, j)$ mi possono servire tre valori:

$$L = \begin{bmatrix} & (i-1, j-1) & (i-1, j) \\ & \swarrow & \uparrow \\ (i, j-1) & \leftarrow & (i, j) \end{bmatrix}$$

LCS(X, Y)

```

1  m = X.length
2  n = Y.length
3  for i = 0 to m
4      L[i, 0] = 0
5  for j = 0 to n
6      L[0, j] = 0
7  for i = 1 to m
8      for j = 1 to n
9          if x_i = y_j
10             L[i, j] = L[i-1, j-1] + 1
11             B[i, j] = '↖'
12          else if L[i-1, j] >= L[i, j-1]
13             L[i, j] = L[i-1, j]
14             B[i, j] = '↑'
15          else
16             L[i, j] = L[i, j-1]
17             B[i, j] = '←'
18  return (L[m, n], B)
```

$\rightarrow$ FORMAX

$$\begin{aligned} \text{MAX}[i, j] &= \\ \text{MAX}[i, j] + \\ L[i-1, j-1] + 1 \end{aligned}$$

$\rightarrow$ ~~FORMALIZZ.~~
 $n^2 \approx m + n$

Esercizio 2 (11 punti) Una *longest common substring* di due stringhe X e Y è una sottostringa di X e di Y di lunghezza massima. Si vuole progettare un algoritmo efficiente per calcolare la lunghezza di una *longest common substring*. Per semplicità si assuma che entrambe le stringhe di input abbiano stessa lunghezza n .

- Qual è la complessità dell'algoritmo esaustivo che analizza tutte le possibili sottostringhe comuni?
- Assumendo di conoscere un algoritmo che determina se una stringa di m caratteri è sottostringa di un'altra stringa di n caratteri in tempo $O(m+n)$, come si può modificare l'algoritmo del punto precedente per renderlo più efficiente?
- Progettare un algoritmo di programmazione dinamica più efficiente di quello del punto precedente. Sono richiesti relazione di ricorrenza sulle lunghezze (senza dimostrazione) e algoritmo bottom-up. (Suggerimento: considerare la lunghezza della *longest common substring* dei prefissi $X_i = \langle x_1, \dots, x_i \rangle$ e $Y_j = \langle y_1, \dots, y_j \rangle$ che termina con x_i e y_j , rispettivamente.)

n^2

Domanda B (6 punti) Si calcoli la lunghezza della longest common subsequence (LCS) tra le stringhe *armo* e *oro*, calcolando tutta la tabella $L[i, j]$ delle lunghezze delle LCS sui prefissi usando l'algoritmo visto in classe.

Soluzione: Si ottiene

	a	r	o
o	0	0	0
a	0	0	0
r	0	0	1
m	0	0	1
o	1	1	2

$\{ \nearrow, \leftarrow \} = \text{MATCH PRIMA}$
 $\{ \nwarrow, \leftarrow \} = \text{MATCH ORA}$

Per calcolare $l(i, j)$ mi possono servire tre valori:

$$L = \begin{bmatrix} (i-1, j-1) & (i-1, j) \\ (i, j-1) & (i, j) \end{bmatrix}$$

Scansione "row-major": riempio la tabella per righe, da sinistra a destra.

Informazione aggiuntiva per costruire la sequenza (vera e propria):

$$b(i, j) = \begin{cases} \nearrow & \text{se } x_i = y_j \\ \leftarrow & \text{se } x_i \neq y_j \text{ e } \max = \text{LCS}(i, j-1) \\ \uparrow & \text{se } x_i \neq y_j \text{ e } \max = \text{LCS}(i-1, j) \end{cases}$$

Esercizio 2 (8 punti) Date due stringhe $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$, si consideri la seguente quantità $\ell(i, j)$, definita per ogni coppia di valori i, j con $0 \leq i \leq m$ e $0 \leq j \leq n$:

$$\ell(i, j) = \begin{cases} 1 & \text{se } i = 0 \text{ o } j = 0 \\ 3\ell(i, j-1) & \text{se } i, j > 0 \text{ e } x_i = y_j \\ 2\ell(i-1, j-1) - \ell(i-1, j) & \text{se } i, j > 0 \text{ e } x_i \neq y_j \end{cases}$$

Si vuole calcolare la quantità $q = \max\{\ell(i, j) : 0 \leq i \leq m, 0 \leq j \leq n\}$.

- Scrivere un algoritmo bottom-up per il calcolo di q .
- Determinare la complessità *esatta* dell'algoritmo, supponendo che le uniche operazioni di costo unitario e non nullo siano i confronti tra caratteri.

$$\begin{aligned} \text{for } i = 1 \text{ to } N \\ L[0, i] = 1 \\ L[i, 0] = 1 \end{aligned}$$

$$n/2 \quad n/2 \rightarrow O(n)$$

$$O(m \cdot n) \dots$$

$$\begin{aligned} \text{for } i = 2 \text{ to } N \\ \text{for } j = N-1 \text{ down to } i \\ \text{if } x_i = x_j \\ \rightarrow 1^0 \\ \text{else} \rightarrow 2^0 \end{aligned}$$

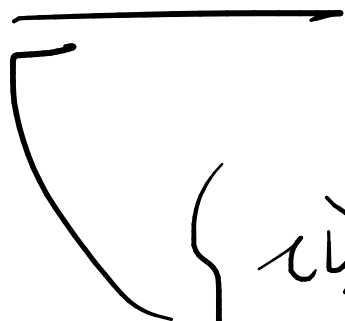
HASH

$$\left(f(h_1) + i \cdot f(h_2) \right) \underline{\text{mod } K}$$

→ DOPPIO HASH

→ CHAINING → LISTE DI TRACCIO

→ LINKED LISTS



~~14~~ ← 13 COLLISIONS!

13 → (14) — IN CODA

ELEMENTO HEAD → COLLISIONS

- HEAD / ALBONI

→ INSERIM.

CANCELLAZIONE.

FUNZIONI!